LAPORAN PRAKTIKUM

POSTTEST 4

ALGORITMA PEMROGRAMAN LANJUT

Heiza Rizki Pratama

2509106019

A'25

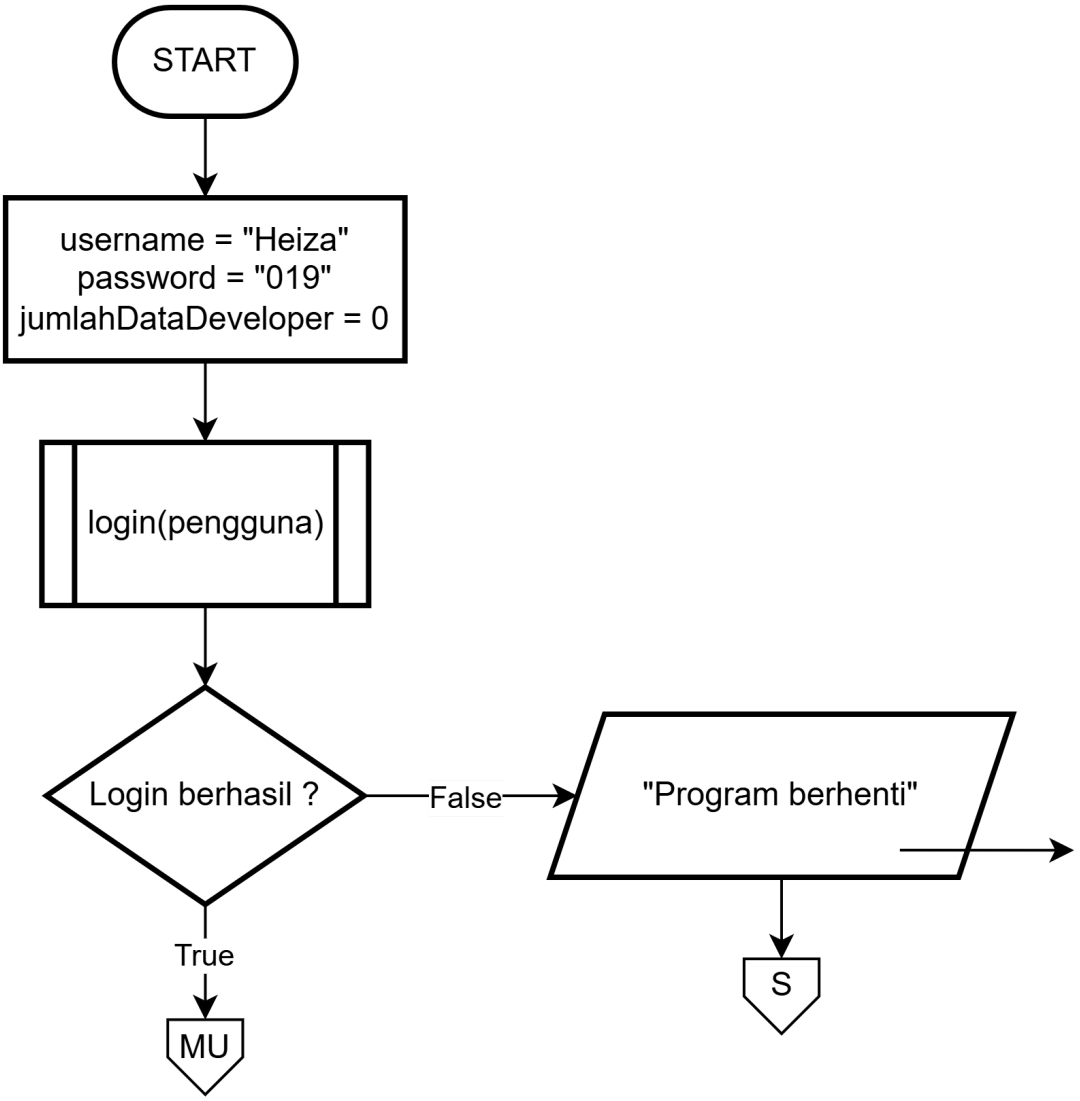
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

SAMARINDA

2026

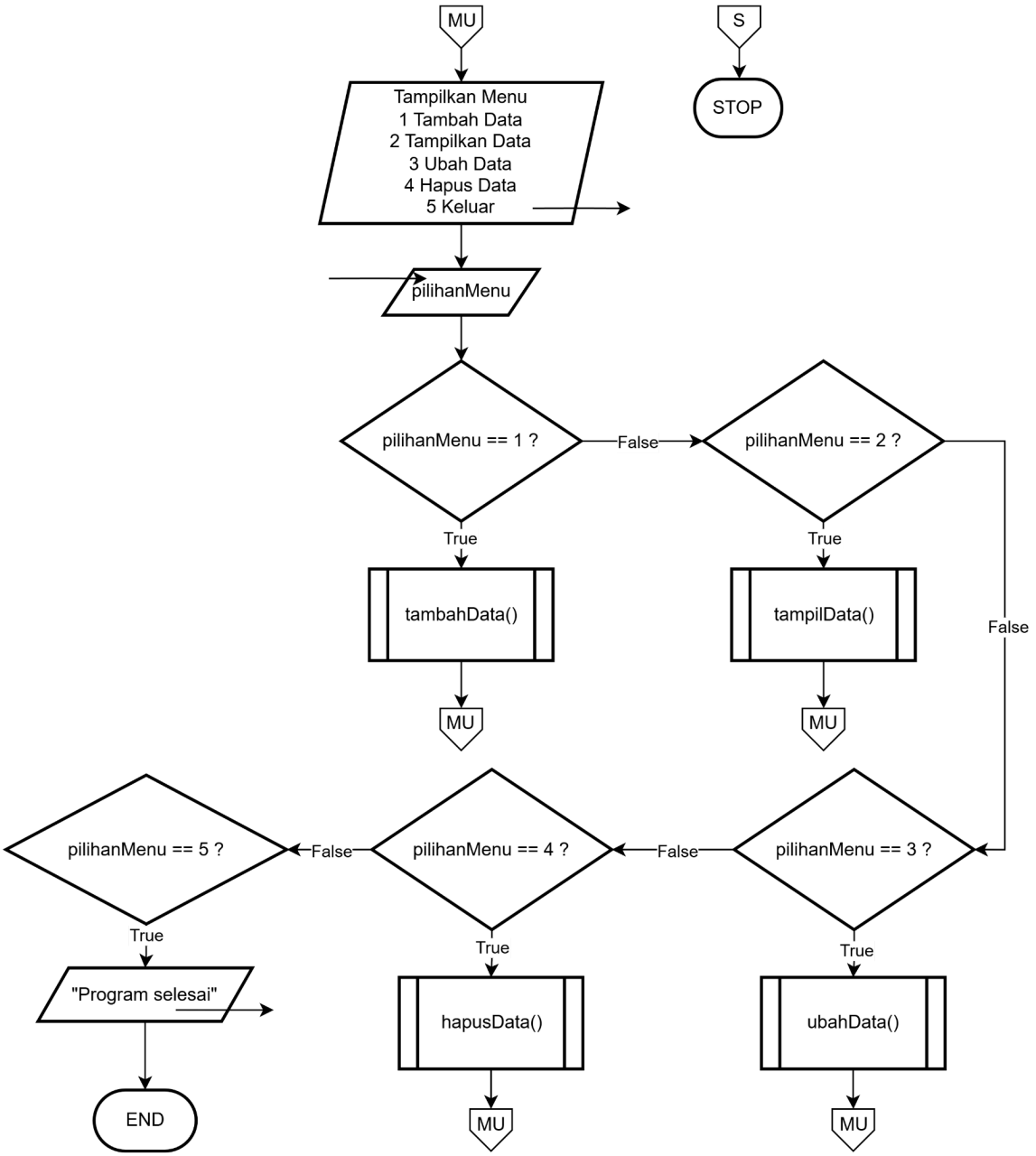
1. Flowchart



Gambar 1.1

Pada Flowchart diatas ini diperlihatkan proses awal program dan proses login pengguna sebelum dapat mengakses sistem.
Program dimulai dari simbol START. Setelah itu program melakukan inisialisasi beberapa variabel, yaitu menetapkan nilai awal username = "Heiza", password = "019", dan jumlahDataDeveloper = 0. Variabel ini digunakan sebagai data awal untuk proses autentikasi dan penyimpanan jumlah data developer.
Selanjutnya program memanggil subproses login(pengguna) yang berfungsi untuk menerima input username dan password dari pengguna. Proses ini menggunakan simbol predefined process, yang menandakan bahwa proses login didefinisikan dalam prosedur atau fungsi terpisah.

Setelah proses login dilakukan, program melakukan percabangan keputusan dengan kondisi "Login berhasil?". Jika kondisi bernilai False (login gagal), maka program menampilkan pesan "Program berhenti" dan alur program menuju simbol STOP, yang berarti program tidak dapat dilanjutkan.
Namun jika kondisi bernilai True (login berhasil), maka alur program akan diarahkan ke connector MU, yang menghubungkan flowchart ini dengan flowchart berikutnya yaitu bagian menu utama program.
Dengan demikian, flowchart ini berfungsi sebagai mekanisme keamanan awal program untuk memastikan hanya pengguna yang memiliki username dan password yang benar yang dapat mengakses sistem.

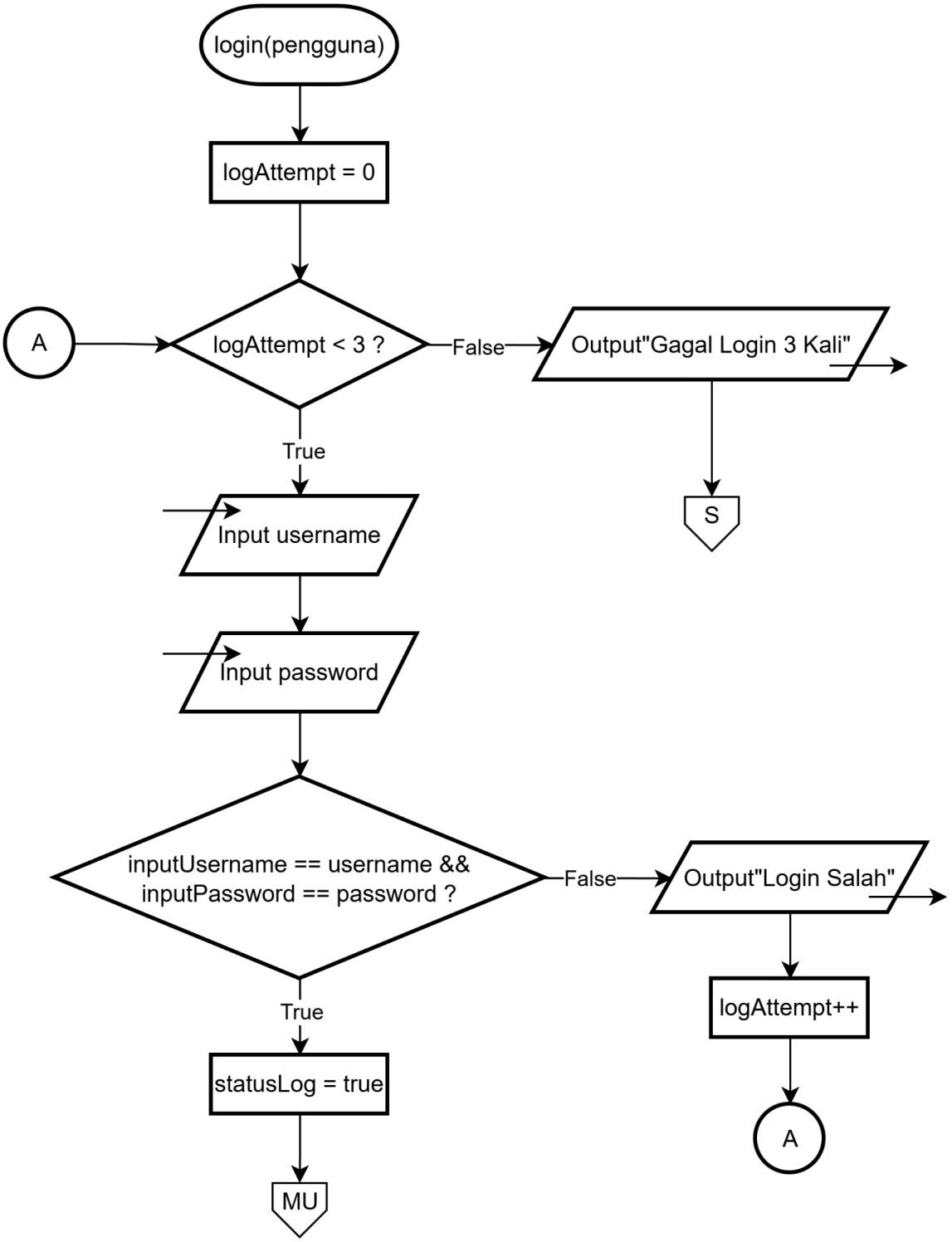


Gambar 1.2

Flowchart kedua menggambarkan alur utama program setelah pengguna berhasil login, yaitu proses pengelolaan data developer melalui menu pilihan.
Proses dimulai dari connector MU yang berasal dari flowchart login. Setelah itu program menampilkan menu utama yang berisi beberapa pilihan, yaitu:

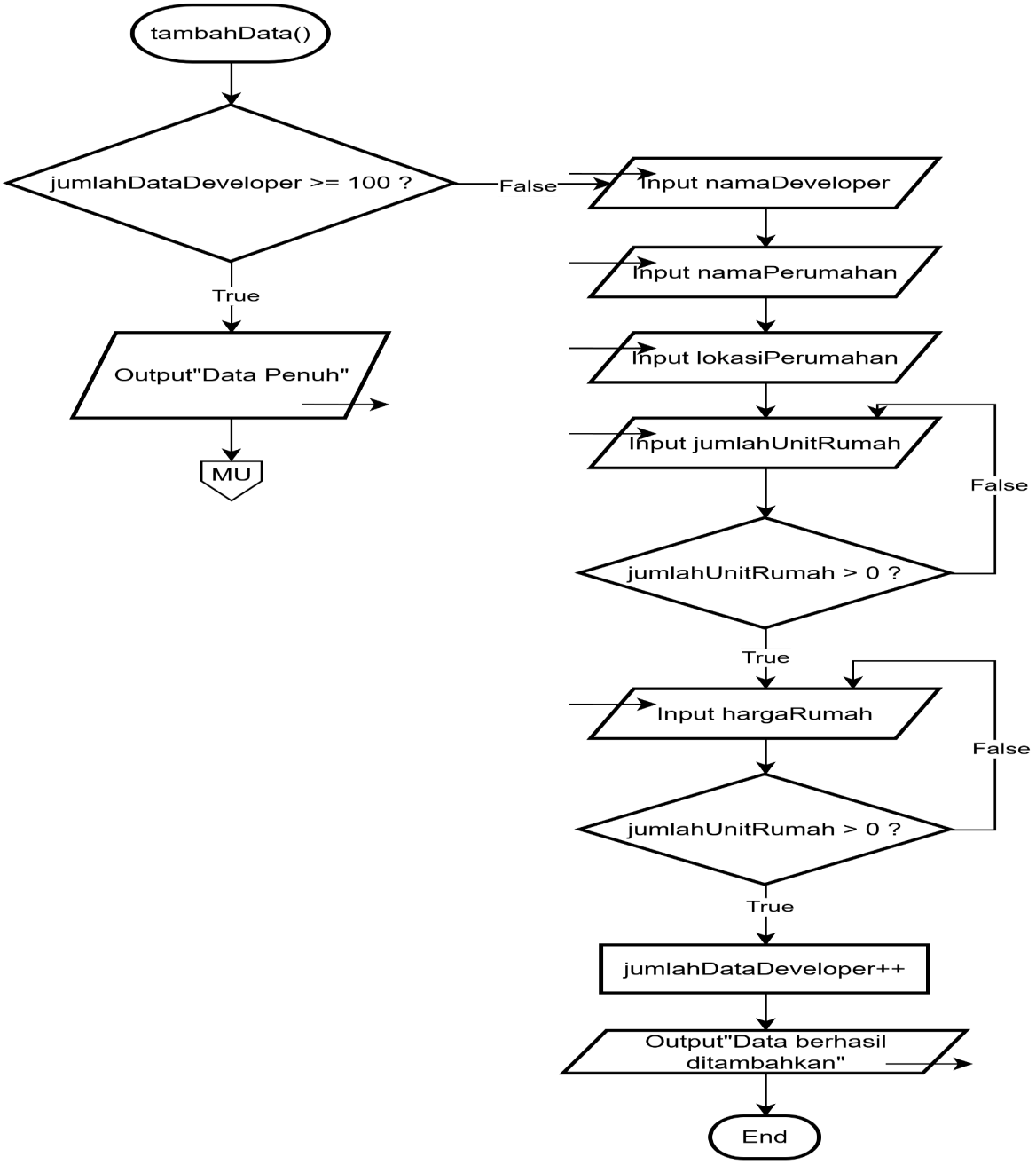
1. Tambah Data
2. Tampilkan Data
3. Ubah Data
4. Hapus Data
5. Keluar

Setelah menu ditampilkan, pengguna diminta memasukkan nilai pilihanMenu sebagai input untuk menentukan proses yang ingin dijalankan.
Program kemudian melakukan serangkaian percabangan keputusan untuk mengecek nilai pilihanMenu.
Jika pilihanMenu == 1, maka program menjalankan fungsi tambahData() yang digunakan untuk menambahkan data developer baru ke dalam sistem. Setelah proses selesai, alur kembali ke menu utama melalui connector MU.
Jika pilihanMenu == 2, maka program menjalankan fungsi tampilData() yang berfungsi menampilkan seluruh data developer yang tersimpan. Setelah data ditampilkan, program kembali ke menu utama.
Jika pilihanMenu == 3, maka program menjalankan fungsi ubahData() untuk melakukan perubahan terhadap data developer yang sudah ada. Setelah perubahan dilakukan, program kembali ke menu utama.
Jika pilihanMenu == 4, maka program menjalankan fungsi hapusData() untuk menghapus data developer tertentu dari sistem. Setelah proses selesai, program kembali lagi ke menu utama.
Terakhir, jika pilihanMenu == 5, maka program menampilkan pesan “Program selesai” dan alur program menuju simbol END, yang menandakan bahwa program telah selesai dijalankan.
Secara keseluruhan, flowchart ini menunjukkan bahwa program menggunakan struktur menu berulang, di mana setelah setiap proses selesai dilakukan, program akan kembali ke menu utama hingga pengguna memilih opsi keluar.



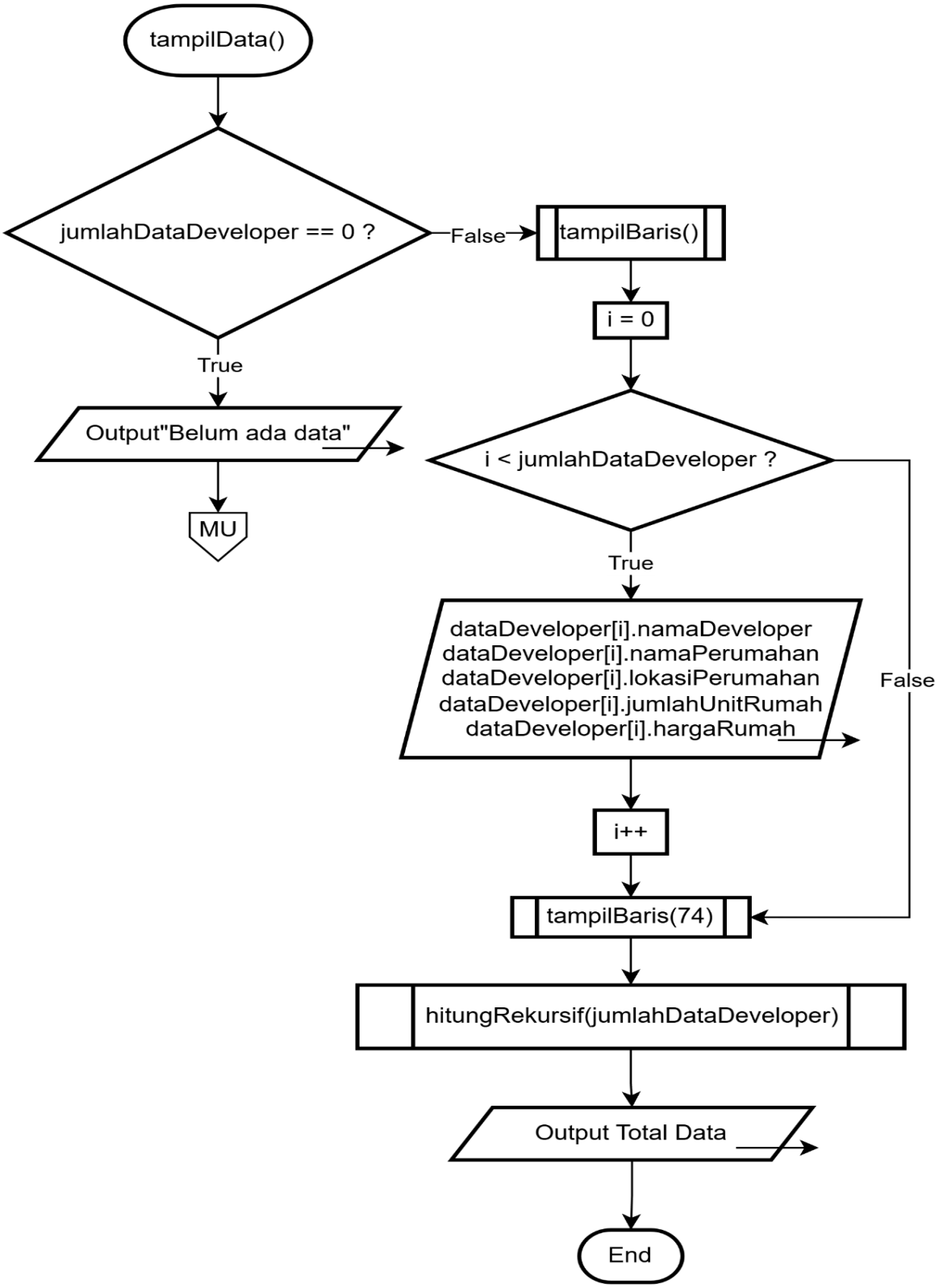
Gambar 1.3

Flowchart login menggambarkan proses autentikasi pengguna dengan batas maksimal 3 kali percobaan. Proses ini menggunakan perulangan dan percabangan untuk menentukan apakah pengguna berhasil masuk atau tidak.



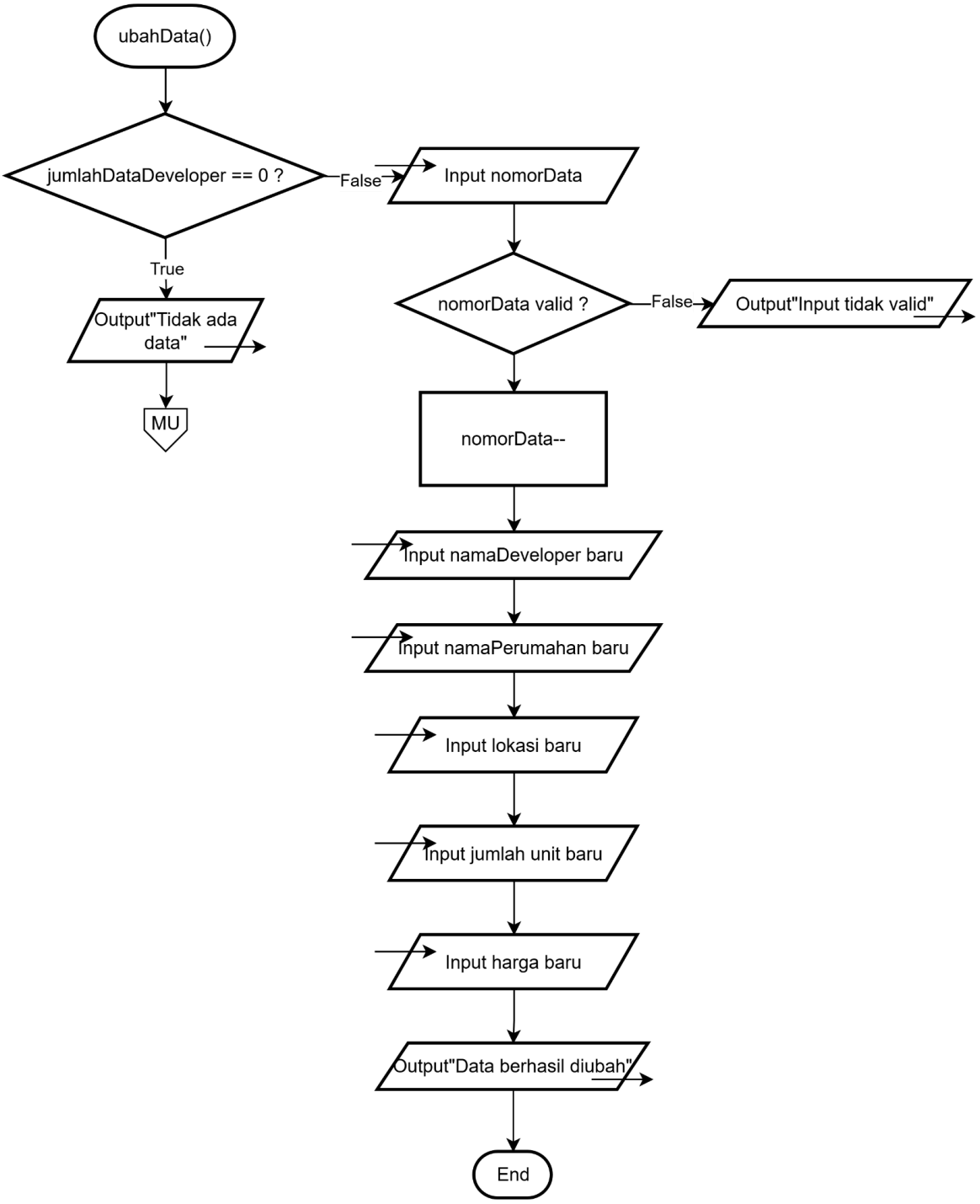
Gambar 1.4

Flowchart tambah data digunakan untuk menambahkan data perumahan ke dalam array dengan batas maksimal 100 data serta dilengkapi validasi input.



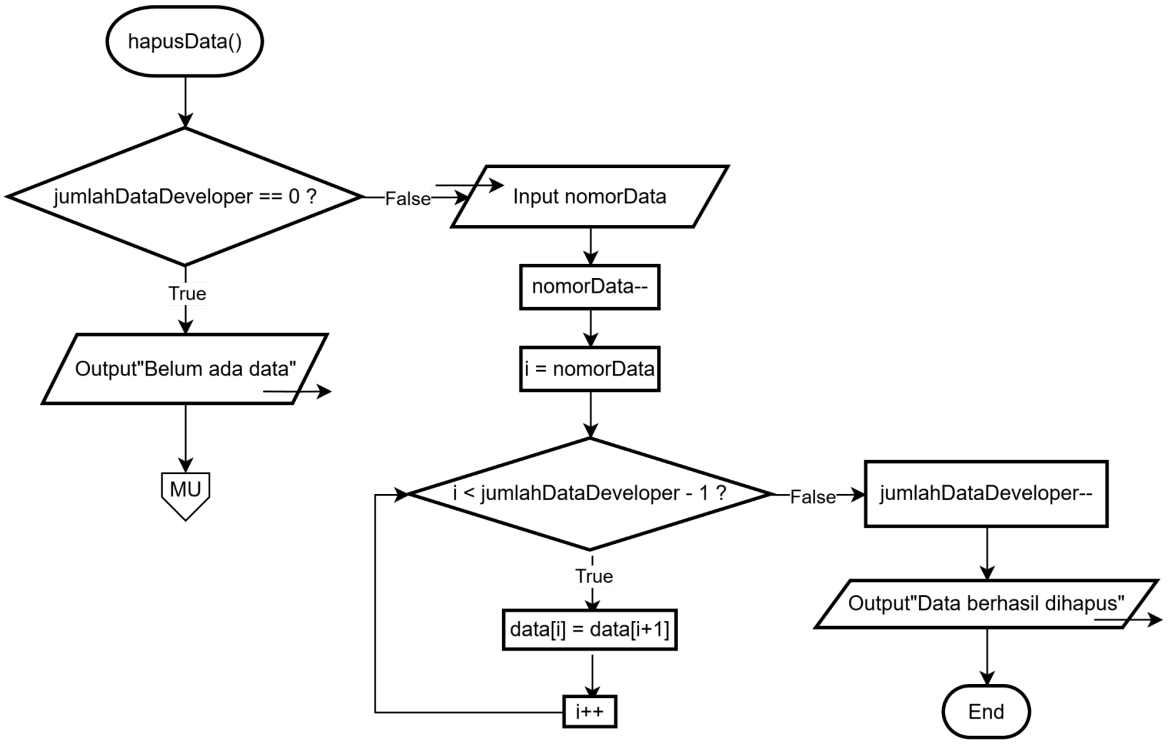
Gambar 1.5

Flowchart tampil data digunakan untuk menampilkan seluruh data developer/perumahan yang tersimpan, serta menampilkan total jumlah data.



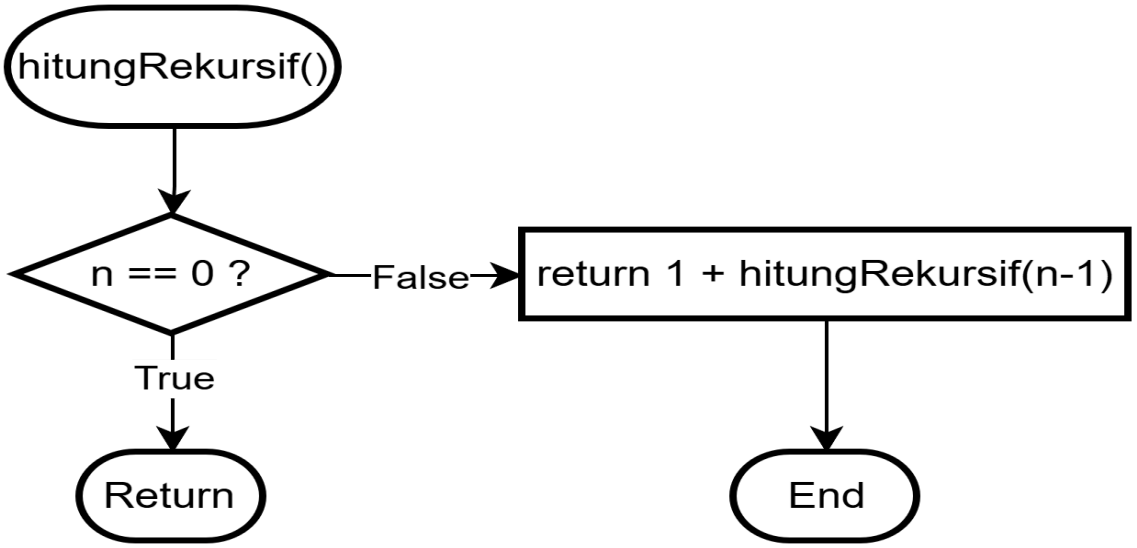
Gambar 1.6

Flowchart ubah data digunakan untuk mengubah data perumahan berdasarkan nomor data yang dipilih oleh user.



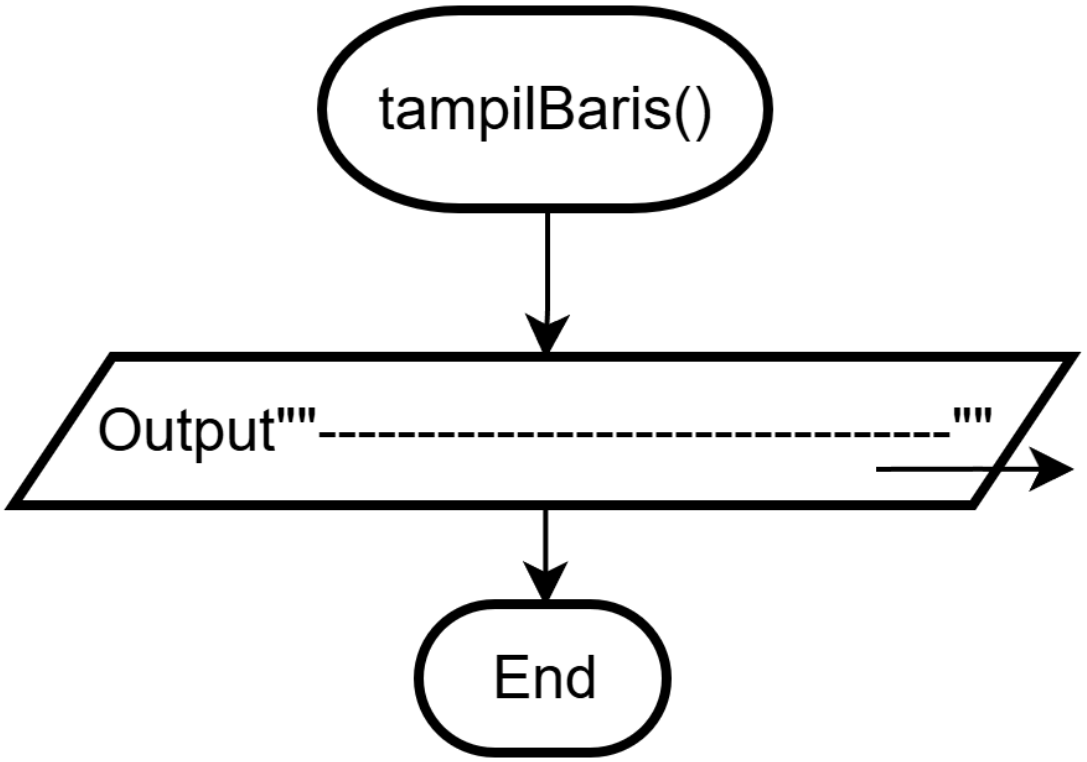
Gambar 1.7

Flowchart hapus data digunakan untuk menghapus data perumahan berdasarkan nomor yang dipilih, dengan cara menggeser elemen array dan mengurangi jumlah data.



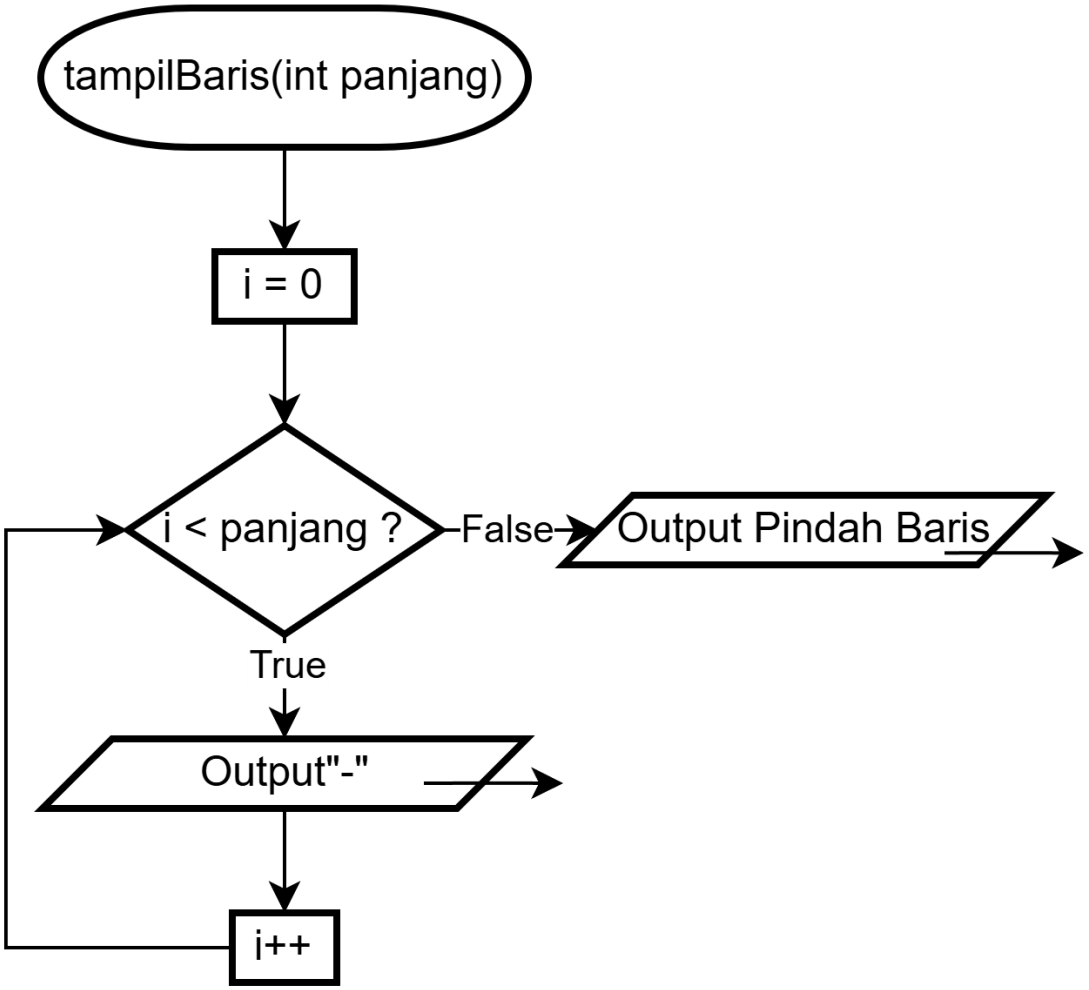
Gambar 1.8

Flowchart hitungRekursif() digunakan untuk menghitung jumlah data secara rekursif, yaitu dengan memanggil fungsi itu sendiri hingga mencapai kondisi dasar.



Gambar 1.9

Menampilkan garis pemisah panjang secara langsung.



Gambar 1.10

Menampilkan garis pemisah sesuai jumlah (panjang) dengan perulangan.

2. Deskripsi Singkat Program

Program ini merupakan Sistem Manajemen Data Developer Perumahan yang dibuat menggunakan bahasa pemrograman C++. Program diawali dengan proses login pengguna yang mengharuskan pengguna memasukkan username dan password yang telah ditentukan sebelumnya. Pengguna diberikan maksimal tiga kali percobaan login. Jika login gagal sebanyak tiga kali, maka program akan berhenti.

Setelah berhasil login, pengguna akan masuk ke menu utama yang menyediakan beberapa fitur untuk mengelola data perumahan yang dikembangkan oleh developer. Program ini menerapkan konsep CRUD (Create, Read, Update, Delete) pada data perumahan yang disimpan dalam bentuk array struktur.

Fitur-fitur yang tersedia dalam program ini meliputi:

1. Tambah Data Perumahan (Create) untuk menambahkan data developer dan perumahan baru.
2. Tampilkan Data Perumahan (Read) untuk menampilkan seluruh data perumahan yang telah disimpan.
3. Ubah Data Perumahan (Update) untuk memperbarui data perumahan yang sudah ada.
4. Hapus Data Perumahan (Delete) untuk menghapus data perumahan tertentu dari daftar.
5. Keluar (Exit) untuk mengakhiri program.

Program ini juga dilengkapi dengan validasi input untuk memastikan data yang dimasukkan pengguna sesuai dengan format yang diharapkan, seperti memastikan jumlah unit rumah dan harga rumah bernilai positif.

3. Source Code

```
struct User{
    string username;
    string password;
};

struct Developer{
    string namaDeveloper;
    string namaPerumahan;
    string lokasiPerumahan;
    int jumlahUnitRumah;
    long hargaRumah;
};

void tampilBaris(){

cout<<"-----------------------------------------------------
----------------------"<<endl;
}

void tampilBaris(int panjang){
    for(int i=0;i<panjang;i++){
        cout<<"-";
    }
    cout<<endl;
}
int hitungRekursif(int jumlahDataDeveloper){
```

```
    if(jumlahDataDeveloper == 0){
        return 0;
    }
    return 1 + hitungRekursif(jumlahDataDeveloper-1);
}
```

```
//Login (pointer)
bool login(User *pengguna){
    string inputUsername;
    string inputPassword;
    int logAttempt = 0;
    bool statusLog = false;

    while(logAttempt < 3){

        system("cls");
        cout<<"===== LOGIN SISTEM =====<<endl;

        cout<<"Masukkan Username (Nama): ";
        cin>>inputUsername;

        cout<<"Masukkan Password (NIM): ";

        inputPassword = "";
        char ch;

        while((ch = _getch()) != 13){
            if(ch == 8){
                if(!inputPassword.empty()){
                    cout<<"\b \b";
                    inputPassword.pop_back();
                }
            }
            else{
                cout<<"*";
                inputPassword += ch;
            }
        }

        cout<<endl;
```

```
        if(inputUsername == pengguna->username && inputPassword ==
pengguna->password){
            statusLog = true;
            break;
        }
        else{
            cout<<"Login Salah"<<endl;
            logAttempt++;
            system("pause");
        }
    }

    if(statusLog == false){
        cout<<"Gagal login 3 kali"<<endl;
    }

    return statusLog;
}
```

```
//Tambah Data (pointer)
void tambahData(Developer *dataDeveloper, int
*jumlahDataDeveloper){

    system("cls");

    cout<<"=== TAMBAH DATA PERUMAHAN YANG DIKEMBANGKAN
==="<<endl;

    if(*jumlahDataDeveloper >= 100){
        cout<<"Data perumahan sudah penuh (maksimal 100
data)."<<endl;
        system("pause");
    }
    else{
```

```
        cin.ignore(1000,'\n');

        cout<<"Nama Developer : ";

getline(cin,dataDeveloper[*jumlahDataDeveloper].namaDevelop
er);

        cout<<"Nama Perumahan : ";

getline(cin,dataDeveloper[*jumlahDataDeveloper].namaPerumah
an);

        cout<<"Lokasi Perumahan : ";

getline(cin,dataDeveloper[*jumlahDataDeveloper].lokasiPerum
ahan);

        cout<<"Jumlah Unit Rumah : ";


while(!(cin>>dataDeveloper[*jumlahDataDeveloper].jumlahUnit
Rumah) ||
dataDeveloper[*jumlahDataDeveloper].jumlahUnitRumah <=0){
            cin.clear();
            cin.ignore(1000,'\n');
            cout<<"Input tidak valid. Masukkan angka
positif: ";
        }

        cout<<"Harga Rumah : ";


while(!(cin>>dataDeveloper[*jumlahDataDeveloper].hargaRumah
) || dataDeveloper[*jumlahDataDeveloper].hargaRumah <=0){
            cin.clear();
            cin.ignore(1000,'\n');
            cout<<"Input tidak valid. Masukkan angka
```

```
positif: ";
        }

        (*jumlahDataDeveloper)++;

        cout<<"Data perumahan berhasil ditambahkan"<<endl;

        system("pause");
    }
}
```

```
//Ubah Data
void ubahData(Developer dataDeveloper[], int
jumlahDataDeveloper){

    system("cls");

    int nomorData;

    cout<<"=== UBAH DATA PERUMAHAN ==="<<endl;

    if(jumlahDataDeveloper == 0){
        cout<<"Tidak ada data."<<endl;
        system("pause");
    }
    else{

        cout<<"Masukkan nomor data (1 -
"<<jumlahDataDeveloper<<") : ";
        cin>>nomorData;

        if(cin.fail() || nomorData < 1 || nomorData >
jumlahDataDeveloper){
            cin.clear();
            cin.ignore(1000,'\n');
            cout<<"Input tidak valid"<<endl;
```

```
            system("pause");
        }
        else{

            nomorData--;

            cin.ignore(1000,'\n');

            cout<<"Nama Developer Baru : ";

getline(cin,dataDeveloper[nomorData].namaDeveloper);

            cout<<"Nama Perumahan Baru : ";

getline(cin,dataDeveloper[nomorData].namaPerumahan);

            cout<<"Lokasi Baru : ";

getline(cin,dataDeveloper[nomorData].lokasiPerumahan);

            cout<<"Jumlah Unit Baru : ";
            cin>>dataDeveloper[nomorData].jumlahUnitRumah;

            cout<<"Harga Baru : ";
            cin>>dataDeveloper[nomorData].hargaRumah;

            cout<<"Data berhasil diubah"<<endl;
            system("pause");
        }
    }
}
```

```
//Hapus Data (pointer)
void hapusData(Developer *dataDeveloper, int
*jumlahDataDeveloper){

    system("cls");
```

```
    int nomorData;

    cout<<"=== HAPUS DATA ==="<<endl;

    if(*jumlahDataDeveloper == 0){
        cout<<"Belum ada data."<<endl;
        system("pause");
    }
    else{

        cout<<"Masukkan nomor data: ";
        cin>>nomorData;

        nomorData--;

        for(int i = nomorData; i <
(*jumlahDataDeveloper)-1; i++){
            dataDeveloper[i] = dataDeveloper[i+1];
        }

        (*jumlahDataDeveloper)--;

        cout<<"Data berhasil dihapus"<<endl;

        system("pause");
    }
}
```

```
int main(){

    User pengguna;

    pengguna.username = "Heiza";
    pengguna.password = "019";
```

```
    if(login(&pengguna) == false){
        return 0;
    }

    Developer dataDeveloper[100];
    int jumlahDataDeveloper = 0;
    int pilihanMenu;

    do{

        system("cls");

        cout<<"===== SISTEM MANAJEMEN =====">>endl;
        cout<<"1. Tambah Data"<<endl;
        cout<<"2. Tampilkan Data"<<endl;
        cout<<"3. Ubah Data"<<endl;
        cout<<"4. Hapus Data"<<endl;
        cout<<"5. Keluar"<<endl;

        cout<<"Pilih Menu : ";
        cin>>pilihanMenu;

        if(pilihanMenu == 1){
            tambahData(dataDeveloper,
&jumlahDataDeveloper);
        }
        else if(pilihanMenu == 2){
            tampilData(dataDeveloper, jumlahDataDeveloper);
        }
        else if(pilihanMenu == 3){
            ubahData(dataDeveloper, jumlahDataDeveloper);
        }
        else if(pilihanMenu == 4){
            hapusData(dataDeveloper, &jumlahDataDeveloper);
        }

    }while(pilihanMenu != 5);
```

```
    cout<<"Program selesai."<<endl;
}
```

Struct User digunakan untuk menyimpan data login yaitu username dan password. Struct Developer digunakan untuk menyimpan data perumahan yang terdiri dari nama developer, nama perumahan, lokasi perumahan, jumlah unit rumah, dan harga rumah.

DataDeveloper[100]= Array untuk menyimpan maksimal 100 data developer jumlahDataDeveloper=Menyimpan jumlah data yang sudah diinput, Digunakan untuk mengetahui posisi data terakhir.

A. Penjelasan Fungsi-Fungsi Program

1. Fungsi login()

Fungsi ini digunakan untuk memverifikasi pengguna sebelum masuk ke sistem. Fungsinya:

1. Menyimpan username dan password yang sudah ditentukan di dalam program.
2. Meminta pengguna memasukkan username dan password.
3. Password disembunyikan menggunakan karakter *.
4. Memberikan maksimal 3 kali percobaan login.
5. Jika login berhasil maka fungsi akan mengembalikan nilai true.
6. Jika gagal 3 kali maka program akan keluar dari sistem.

2. Fungsi tambahData()

Fungsi ini digunakan untuk menambahkan data developer perumahan ke dalam sistem. Fungsinya:

1. Meminta pengguna menginput nama developer.
2. Meminta input nama perumahan.
3. Meminta input lokasi perumahan.
4. Meminta input jumlah unit rumah.
5. Meminta input harga rumah.
6. Data yang dimasukkan akan disimpan ke dalam array dataDeveloper.
7. Setelah data disimpan, jumlah data (jumlahDataDeveloper) akan bertambah satu.

3. Fungsi tampilkanRekursif()

Fungsi ini digunakan untuk menampilkan seluruh data developer menggunakan metode rekursif. Fungsinya:

1. Menampilkan data developer satu per satu dalam bentuk tabel.
2. Fungsi akan memanggil dirinya sendiri untuk menampilkan data berikutnya.
3. Proses berhenti ketika semua data sudah ditampilkan.

4. Fungsi tampilkanData()

Fungsi ini digunakan untuk menampilkan semua data developer yang tersimpan. Fungsinya:

1. Mengecek apakah data developer masih kosong atau tidak.
2. Jika kosong maka akan menampilkan pesan belum ada data.
3. Jika ada data maka akan menampilkan header tabel seperti:
    a. nomor
    b. developer
    c. perumahan
    d. lokasi
    e. unit
    f. harga
4. Setelah itu fungsi akan memanggil fungsi rekursif untuk menampilkan semua data.

5. Fungsi tampilkanData(int index)

Fungsi ini merupakan fungsi overload dari tampilkanData(). Fungsinya:

1. Digunakan untuk menampilkan satu data developer saja.
2. Data yang ditampilkan ditentukan berdasarkan index yang diberikan.

6. Fungsi ubahData()

Fungsi ini digunakan untuk mengubah data developer yang sudah ada. Fungsinya:

1. Meminta pengguna memasukkan nomor data yang ingin diubah.
2. Program kemudian meminta data baru seperti:
    a. nama developer
    b. nama perumahan
    c. lokasi
    d. jumlah unit
    e. harga
3. Data lama akan diganti dengan data baru yang diinput.

7. Fungsi hapusData()

Fungsi ini digunakan untuk menghapus data developer dari daftar. Fungsinya:

1. Meminta pengguna memasukkan nomor data yang ingin dihapus.
2. Data yang dipilih akan dihapus dari array.
3. Data setelahnya akan digeser ke depan agar tidak ada data kosong.
4. Jumlah data developer akan dikurangi satu.

8. Fungsi main()

Fungsi ini merupakan fungsi utama yang menjalankan program. Fungsinya:

1. Memanggil fungsi login() terlebih dahulu.
2. Jika login berhasil maka program akan menampilkan menu utama.
3. Menu berisi pilihan:
    a. Tambah Data
    b. Tampilkan Data
    c. Ubah Data
    d. Hapus Data
    e. Keluar
4. Program menggunakan perulangan sehingga menu akan terus muncul sampai pengguna memilih keluar.

4. OUTPUT PROGRAM

```
===== LOGIN SISTEM =====
Masukkan Username (Nama): Heiza
Masukkan Password (NIM): ***
```

Login

```
===== SISTEM MANAJEMEN DATA DEVELOPER PERUMAHAN =====
1. Tambah Data Perumahan
2. Tampilkan Data Perumahan
3. Ubah Data Perumahan
4. Hapus Data Perumahan
5. Keluar
Pilih Menu :
```

Menu Utama

```
=== TAMBAH DATA PERUMAHAN YANG DIKEMBANGKAN ===
Nama Developer : Lumpus
Nama Perumahan : L corp
Lokasi Perumahan : Jl. setiabudi
Jumlah Unit Rumah : 189
Harga Rumah : 179000000
Data perumahan berhasil ditambahkan
Press any key to continue . . .
```

Create Data Perumahan

```
=== DATA PERUMAHAN YANG SEDANG DIKEMBANGKAN ===
--------------------------------------------------------------------------
No | Developer | Perumahan | Lokasi | Unit Rumah | Harga Rumah
--------------------------------------------------------------------------
1 | Lumpus | L corp | Jl. setiabudi | 189 | 179000000
--------------------------------------------------------------------------
Total Data : 1
Press any key to continue . . .
```

Read Data Perumahan

```
=== UBAH DATA PERUMAHAN ===
Masukkan nomor data yang ingin diubah (1 - 1) : 1
Nama Developer Baru : Luminous
Nama Perumahan Baru : Gugugaga
Lokasi Perumahan Baru : Jl. Lingai Jati
Jumlah Unit Rumah Baru : 190
Harga Rumah Baru : 400000000
Data berhasil diperbarui
Press any key to continue . . .
```

Update Data Perumahan

```
=== HAPUS DATA PERUMAHAN ===
Masukkan nomor data yang ingin dihapus (1 - 1) : 1
Data perumahan berhasil dihapus
Press any key to continue . . .
```

Delete Data Perumahan

5. Langkah-langkah GIT

```
PS C:\Users\Lucius\Documents\Praktikum-Semester-2\APL> git init
Initialized empty Git repository in C:/Users/Lucius/Documents/Praktikum-Semester-2/APL/.git/
PS C:\Users\Lucius\Documents\Praktikum-Semester-2\APL> git add .
PS C:\Users\Lucius\Documents\Praktikum-Semester-2\APL> git commit -m "First Upt ✅"
[master (root-commit) 2c92f6d] First Upt ✅
 2 files changed, 82 insertions(+)
 create mode 100644 Praktikum-APL/post-test/2509106019-HeizaRizki-PT-1.cpp
 create mode 100644 Praktikum-APL/post-test/2509106019-HeizaRizki-PT-1.exe
PS C:\Users\Lucius\Documents\Praktikum-Semester-2\APL> git branch -M main
PS C:\Users\Lucius\Documents\Praktikum-Semester-2\APL> git remote add origin https://github.com/01001100-KMS/APL.git
PS C:\Users\Lucius\Documents\Praktikum-Semester-2\APL> git push -u origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (6/6), 48.00 KiB | 2.00 MiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/01001100-KMS/APL.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\Lucius\Documents\Praktikum-Semester-2\APL>
```

5.1 GIT Add

```
PS C:\Users\Lucius\Documents\New folder> git add .
```

Fungsinya**:** Memilih file yang sudah diubah untuk masuk ke staging **area** (daftar siap commit). Tanpa git add, perubahan tidak akan ikut tersimpan saat git commit.

Contoh:
git add index.html
→ hanya file index.html yang siap di-commit.
git add .
→ semua file yang berubah akan masuk staging area.

5.2 GIT Commit

```
PS C:\Users\Lucius\Documents\New folder> git commit -m "ini commit"
[main 94354ce] ini commit
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Fungsinya: Menyimpan perubahan yang sudah dipilih (staging area*) ke dalam riwayat repository*

Commit ini ibarat checkpoint atau simpan versi dari proyek. Biasanya commit disertai pesan (-m) agar jelas maksud perubahannya. Contoh:

commit - git m "Menambahkan fitur login"

5.3 GIT Push

Fungsinya: Mengirim perubahan (commit) yang ada di repository lokal ke repository remote (misalnya GitHub, GitLab, Bitbucket). Supaya bisa push, biasanya harus sudah git remote add origin <url> terlebih dahulu.

Contoh:
 git push origin main

→ Mengirim commit lokal ke branch main di repository remote bernama origin.